

SMART IOT IRRIGATION & PLANT-CARE SYSTEM

PROJECT VERDE

The plant that waters itself — and talks to AI.

5 Aarav Choudhary • Anuj

2

2/s

₹1,890

Class X — a ₹1,890 system with a ₹8,000+ commercial-feel

SENSORS

MCUs

CLOUD CALLS

TOTAL COST

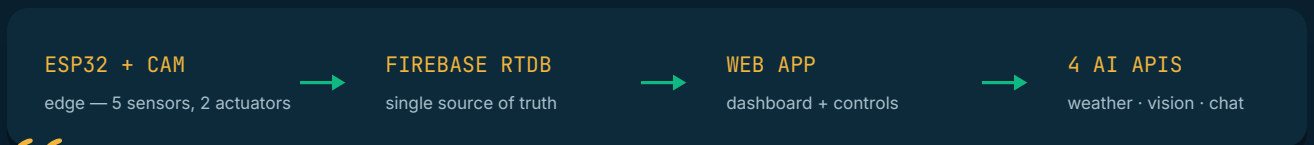
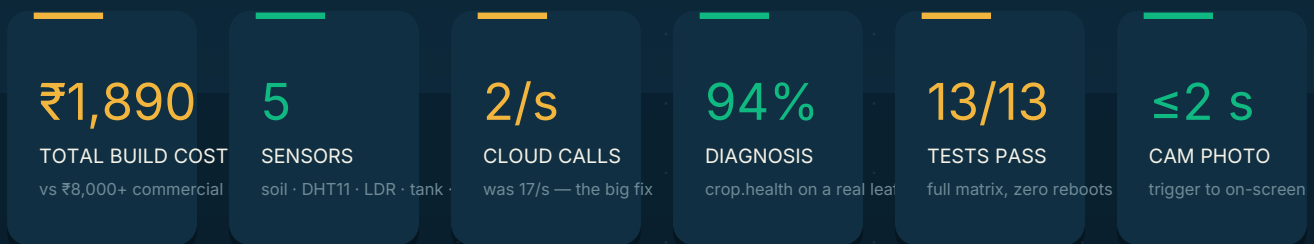
The Whole Story in 60 Seconds

Everything a judge needs before the demo even starts.

Most plants don't die from neglect — they die from **no information**. Nobody knows in real time how dry the soil is, whether the tank is empty, or whether rain is coming. Project Verde removes that blind spot: a three-tier IoT system where the plant **tells us what it needs**, and the system acts on its own.

At the edge, an **ESP32 WROOM-32** reads 5 sensors and drives 2 actuators while an **ESP32-CAM** acts as the plant's eyes. Every second, one bundled HTTPS call carries 10 live metrics to a **Firestore Realtime Database** — the single source of truth. On top sits a single-file web app: live dashboard, weather with automatic rain-override, a Plant Doctor that identifies diseases at **94% confidence**, and AI assistants that chat about the same photo you're looking at.

Total build cost: **≈ ₹1,890**. All software and APIs are on free tiers. Everything is student-built, student-understood, and demo-ready — with a genuinely fun bug story inside (17 cloud calls per second → 2).



“

A ₹1,890 student project that behaves like a funded startup product.

— the goal, stated plainly

Contents

Hyperlinked — click any entry to jump. Bookmarks in your PDF reader too.

01	The Whole Story in 60 Seconds	2	11	Web App — Plant Doctor & AI	13
02	Why — The Problem	4	12	AI & APIs — Accuracy Notes	14
03	How It Works — Architecture	5	13	Features — All Live	15
04	Hardware — The Brain & Eyes	6	14	Testing — 13/13 Matrix	16
05	Hardware — Wiring & Power	7	15	Troubleshooting Journal	17
06	Firmware — The Brain	8	16	Cost & Sustainability	18
07	The Big Bug (17 → 2 calls)	9	17	Future Scope	19
08	ESP32-CAM — The Eyes	10	18	Judge Tour Script	20
09	Cloud — Firebase Schema	11	19	Conclusion	21
10	Web App — Dashboard & Weather	12			



How this document works

Short pages, honest numbers, zero walls of text.
Every page is skimmable in under 10 seconds — the
demo script at the end turns it into a live tour.



The numbers are real

Rs. 1,890 · 5 sensors · 2 MCUs · 17→2 calls/s · 94%
diagnosis · 8 MHz XCLK. Every figure in here
matches the build log exactly.

Why? Because Plants Die of Ignorance

Not from neglect. From a lack of information.

Urban families forget to water plants. Or they over-water them out of guilt. Both kill the plant — and neither is really the problem. The problem is that nobody **knows**: how dry is the soil right now? Is the tank empty? Is rain on the way?



FORGOTTEN

Life gets busy. The plant is the first thing to slip — silently, for days.



OVER-WATERED

Guilt-watering drowns roots. Enthusiasts kill plants with kindness.



NO VISIBILITY

No real-time soil, tank, or weather data — decisions made blind.



PRICE WALL

Commercial smart kits start at ₹8,000+, and still lack cameras and AI.

THE COST OF IGNORANCE

Commercial smart-garden kits: ₹8,000+, no camera, no AI, closed hardware a student can't open or understand. The gap isn't money — it's information. So we built the information.



A plant doesn't need a gardener with a schedule. It needs a gardener with sensors.

— Project Verde, in one line

Three Tiers, One Truth

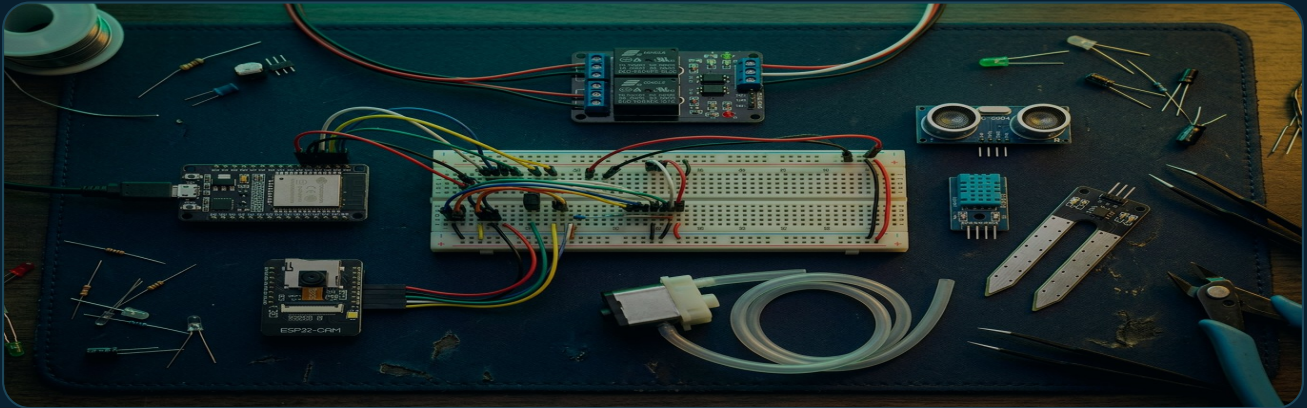
Edge sensors → one cloud → an experience you can actually read.

Every second, the ESP32 bundles 10 live metrics into one write to `/sensors` and reads back 9 control keys from `/controls` in one call. The web app polls the same database over REST. Both sides see the exact same truth — no sync layers, no conflicts, one source.



The Brain & The Eyes

Two ESP32 boards, five sensors, two actuators — one very patient plant.



The whole bench: ESP32 brain, ESP32-CAM eyes, sensors, relay, pump — under ₹1,900.

MODULE	ESP32 PIN	ROLE
LM393 soil moisture	AO → GPIO34	% soil wetness — power-gated 15 ms reads, anti-corrosion temperature + humidity
DHT11	DATA → GPIO4	
LDR module	AO → GPIO35	ambient light → "dark" detection
HC-SR04 ultrasonic	TRIG → GPIO18 · ECHO → GPIO19	water-tank level, 5-point filter
2-ch relay	IN1 → GPIO5 (active-LOW)	switches the 5 V water pump
UV grow LED	GPIO12 (active-HIGH, 220 Ω)	photosynthetic light
ESP32-CAM OV2640	own board + MB programmer	SVGA photos → cloud, on demand

5

SENSORS

2

ACTUATORS

2

MCUS

5V/2A

POWER SUPPLY

Wiring & The Power Wars

Every pin, every lesson — the stuff that actually took the longest.

The wiring diagram above is the real, final revision — each of those lines is a wire we physically pulled, swore at, and re-pulled. The interesting part is below the schematic: **power design is where this project was won.**



5 V / 2 A — never USB-PD

A 67 W laptop charger starved the board: PD needs a handshake chip the ESP32 lacks, so it delivered ~0 mA. A plain phone adapter fixed it.



1000 μ F across the rail

The electrolytic capacitor absorbs pump + WiFi current spikes that were crashing brownouts into resets.



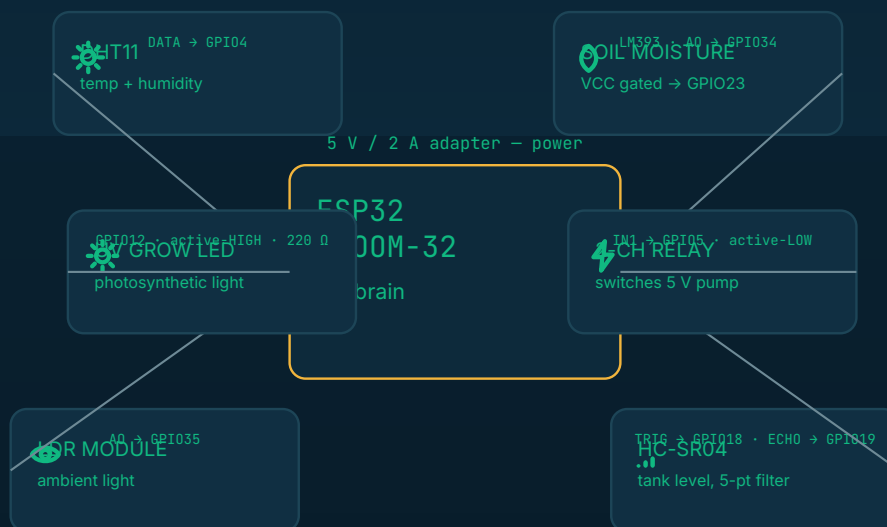
1N4007 flyback diode

Across the pump, it kills the inductive voltage spike every switch-off would otherwise send back into the logic.



Relay isolation

The pump runs on its own 5 V source via relay COM/NO — electrically isolated from the ESP32's power, so motor noise never corrupts sensor reads.



1000 μ F electrolytic

across 5 V/GND — absorbs pump + WiFi spikes

1N4007 flyback diode

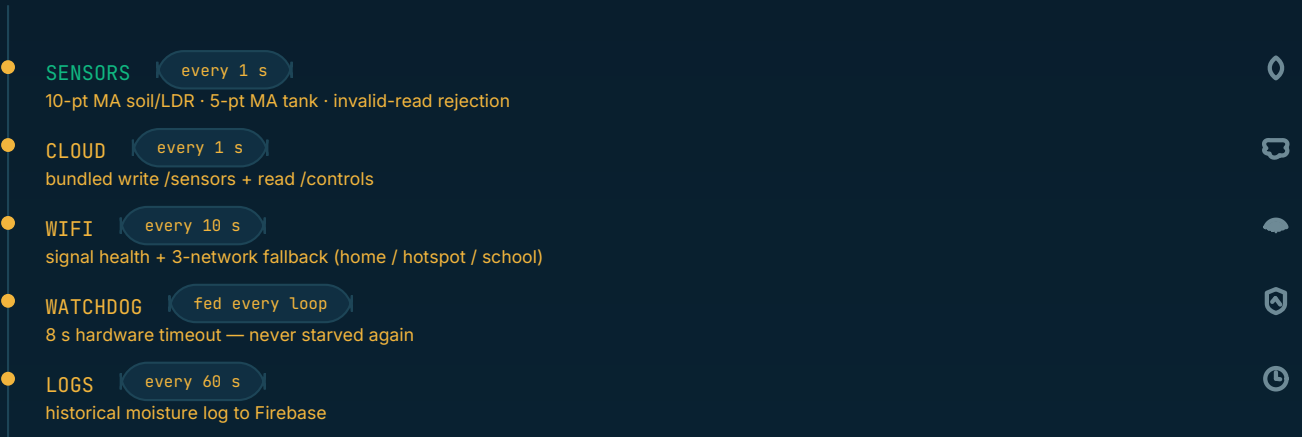
across the pump — kills inductive spikes

Relay isolation

pump on its own 5 V rail via COM/NO

The Brain: Non-Blocking by Design

Code_1_Main_Brain.ino — V3.0.7-FINAL. Nothing blocks. Nothing freezes. Nothing starves.



AUTO LOGIC — ONE LINE

```
pump_ON = moisture < threshold
          AND tank ≥ safe
          AND no rain
```

Manual mode overrides but never skips the tank check — a dry tank can't be pumped.

THRESHOLDS — LIVE FROM THE APP

```
moisture_threshold      35%
tank_threshold          15% (0 = disabled)
light_threshold         35
```

Persisted in NVS flash — survive reboots, adjustable from the app, no reflash.



3-network WiFi fallback

home → hotspot → school. The demo never dies on a captive portal.



8 s hardware watchdog

fed every loop — a reboot now means a real fault, not a stall.



±2% hysteresis

light auto-switch can't flicker at the boundary.



10-pt moving averages

soil/LDR smoothed; tank uses 5-pt + invalid-read rejection — pump splash can't fake an empty tank.

The Bug That Almost Killed AUTO

AUTO mode clicked the pump ON and OFF every ~10 seconds. Here's why — and the fix.

Root cause: 17 blocking Firebase HTTPS calls per second — one per key, per sensor, per second. The network stalled, the 8-second watchdog rebooted the board, and every reboot re-evaluated AUTO with a dry sensor reading. Loop. The pump never stayed on long enough to water anything.

The fix: **JSON bundling** — one write of all 10 metrics to `/sensors` plus one read of 9 control keys from `/controls` per second. Latency dropped ~85%, reboots hit zero, and the pump finally stays ON continuously until the soil reaches threshold. This is the bug that taught us the whole project: **design the data flow before the feature**.

BEFORE — v3.0.6

17 HTTPS calls every second

1 write + 1 read per API key, individually

- Firebase blocks pile up → network stalls
- 8 s watchdog fires → ESP32 reboots
- loop restarts → AUTO mode re-evaluates
- pump clicks ON / OFF every ~10 s

AFTER — v3.0.7-FINAL

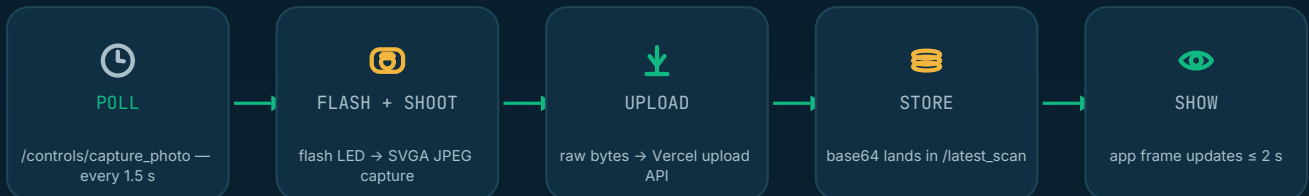
2 bundled calls per second

1 write of 10 metrics + 1 read of 9 keys

- JSON bundling → ~85% less latency
- watchdog fed every loop → 0 reboots
- pump stays ON until threshold reached
- plant actually gets watered

ESP32-CAM: A Photo Every 2 Seconds

Code_2_ESP32_CAM.ino — V3.0.4-FINAL. The plant has a camera now.



8 MHz XCLK

Camera clock throttled from 20 to 8 MHz — RF interference with the WiFi antenna gone.



Sequential boot

Camera first, WiFi 500 ms later — no power surge, no crash 0x20002.



esp_camera_fb_return()

Frame buffer returned instantly — no heap fragmentation after hours.



Poll-triggered

Polls `capture_photo` every 1.5 s → photo on screen in ≤2 s.



The Plant Doctor: photo → diagnosis → AI chat.

The photo pipeline runs independent of the brain — the ESP32-CAM is its own little computer with one job: **see the plant, tell the cloud**. Later pages show what the cloud and AI do with that photo.

Firestore: One Truth, Six Branches

Every byte the system knows lives in one Realtime Database: verde-tech-haha.



Rules that behave

Public read for the demo, validated writes — booleans stay booleans, numbers stay 0–100. ESP32 auth: legacy database secret.



10 metrics, 1 write

sensors/ carries moisture, temp, humidity, light, tank, lux, watchdog status, voltage sag, upload counters — bundled per second.



Controls are shared state

mode, pump, light, thresholds, capture trigger and weather override live in controls/ — written by the app, obeyed by the ESP32.

verde-tech-haha (Realtime Database)

sensors/



moisture · temperature · humidity · light · tank_level · lux · watchdog_status · voltage_sag · successful_uploads · failed_uploads

controls/



manual_mode · pump_state · light_manual_mode · grow_light_state · capture_photo · moisture_threshold · tank_threshold · light_threshold · weather_override

latest_scan/



imageUrl (base64) · status · captured_at · scientificName · diseaseName · probability · treatmentPlan

weather/



city · temp · condition · description · humidity · wind_speed · rain_expected · synced_at

historical_logs/



moisture_log [{ time, moisture }]

actuators/



pump_actual · grow_light_actual · mode

Dashboard & Weather

A single-file HTML app — four pages via the burger menu. No frameworks, no build step.



- ▶ 8 live telemetry tiles with sparklines — hover any tile for a last-10 trend, ▲/▼ arrows
- ▶ All 8 controls + 3 threshold sliders — thresholds persist to NVS, no reflash
- ▶ Weather page — live Delhi conditions, 5-day forecast chips, auto rain-override checked every 3 min
- ▶ Predicted actuator states — the app shows what the ESP32 **will** do before it does it
- ▶ System status strip · toasts · fullscreen demo mode · uptime timer · moisture history chart

Plant Doctor & AI Assistants

The pages that make judges say "wait, this is a school project?"



Plant Doctor: live camera frame, capture button, diagnosis card, and AI chat on the same image.



Plant Doctor

A live camera frame auto-refreshes every ≤ 2 s. Press CAPTURE — or upload a photo — and crop.health identifies the plant and any disease, with probability and a treatment plan. The diagnosis lands back in the same view, and the AI chat sees the same image.



AI Assistants

Two assistants on demand: a Gemini vision chat that reasons over the analysed photo, and an OpenRouter chat that reads live sensor telemetry. Quick prompts get the plant talked about in plain language — "should I water it tonight?"



Tank calibration pane

SET EMPTY / SET FULL in the app — the tank remaps to real millimetres with no reflash.



Camera flip fix

The CAM mounts upside-down on the pot; one toggle flips the image. Physics solved in CSS.



Photo ≤ 2 s

poll → flash → capture → upload → base64 → screen. The whole loop fits inside one camera refresh.



Upload or CAM

Don't have the hardware at hand? Upload any plant photo and the whole pipeline still runs.

Four APIs, Zero Drama

Researched, keyed, and live-tested. With the accuracy notes to prove it.



OpenWeatherMap

live weather + 5-day forecast · key in URL

GET /data/2.5/weather?q=Delhi — ids 2xx/3xx/5xx/6xx set weather_override=1

▲ live-tested: Delhi 35 °C, correct city id 1273294



crop.health (Plant.id)

plant + disease identification · Api-Key header

POST /api/v1/identification with base64 image → crop + disease suggestions

▲ test leaf: nutrient deficiency @94% with treatment plan



Gemini 2.5 Flash

vision chat on the analysed photo · X-goog-api-key (AQ keys)

POST /v1beta/models/gemini-flash-latest:generateContent — inline image + diagnosis + telemetry

▲ gemini-2.5-flash no longer offered to new users → gemini-flash-latest



OpenRouter

sensor chat + vision fallback · Authorization: Bearer sk-or-v1-...

POST /api/v1/chat/completions (OpenAI-compatible) — 8-model text + 5-model vision chains

▲ 435 models; free models rotate → fallback chains never dead-end

TEXT CHAIN — 8 MODELS

sensor-aware chat, OpenAI-compatible

meta-llama · mistral · qwen · deepseek

command-r · gpt-4o-mini ·  gemini · free tier rotation

if model A fails → try B → ... never a dead end

VISION CHAIN — 5 MODELS

sees the same plant photo + diagnosis

gemini-flash-latest (primary)

4 vision-capable fallbacks 

435 models accessible via one key

Features: All Working, All Demo-Ready

Not a roadmap. A checklist. Every item here ran in the real demo.

 1-second heartbeat 10 metrics up, 9 controls down — one bundled call each second.	 AUTO watering moisture < 35% AND tank ≥ 15% AND no rain → pump on, stays on.	 Smart grow light LDR-based, ±2% hysteresis, manual override, threshold from the app.
 Live weather Delhi conditions + 5-day forecast + rain override with countdown.	 Plant Doctor Live CAM frame, capture ≤2 s, 94% diagnosis with treatment plan.	 AI vision chat Gemini sees the same photo, with the diagnosis as context.
 AI sensor chat OpenRouter reads live telemetry and answers in plain language.	 Tank protection AUTO and manual both refuse to run a dry pump. Physics, enforced.	 Moisture history logged every 60 s — watch the watering cycle chart come alive.
 Watchdog + NVS 8 s watchdog, thresholds persisted in flash, reboot-safe.	 Tank calibration SET EMPTY / SET FULL — remap the tank in-app, no reflash.	 Demo mode fullscreen kiosk view with uptime timer — judges love the counter.

Everything above is driven by the same single-file web app — the one the judges will hold on a phone. **Tap. Water. Diagnose. Chat.**

13 Tests, 13 Passes

A 13-point test matrix. Everything green — including a 10-minute watchdog run with 0 reboots.

TEST	RESULT	EVIDENCE
WiFi + boot	✓ PASS	3-network fallback verified
DHT11 breathe test	✓ PASS	stable temp/humidity readings
Soil moisture dunk test	✓ PASS	water → reading jumps correctly
LDR cover test	✓ PASS	dark → light threshold triggers
Ultrasonic hand test	✓ PASS	tank level tracks hand distance
Pump AUTO 120 s	✓ PASS	no glitch, continuous run
OFF at threshold	✓ PASS	pump stops exactly at 35%
Tank lock	✓ PASS	dry tank blocks pumping
Rain override	✓ PASS	weather_override gates AUTO
CAM capture ≤2 s	✓ PASS	trigger → screen
Plant Doctor	✓ PASS	94% diagnosis on test leaf
AI chats + fallbacks	✓ PASS	8-model / 5-model chains
Watchdog 10+ min	✓ PASS	0 reboots, 0 stalls

13/13

TESTS PASSED

full matrix, all green

0

REBOOTS IN 10+ MIN

watchdog fed every loop

94%

BEST DIAGNOSIS

nutrient deficiency, treatment plan

Ten Bugs We Hit. Ten Fixes We Kept.

Honesty is a feature. Every one of these cost us real evenings — and taught us more than any tutorial.

- 01 AUTO pump clicked ON/OFF every ~10 s
→ 17 blocking HTTPS calls/s → JSON bundling: 2 calls/s
- 02 Camera probe 0x106 — not found
→ FPC ribbon unseated → reseal, gold-side down, power cycle
- 03 PSRAM not found
→ Weak power → dedicated 5 V / 2 A adapter
- 04 Boot crash 0x20002
→ Camera + WiFi power surge → sequential boot, camera first
- 05 RF interference killed WiFi
→ 20 MHz XCLK → throttled to 8 MHz
- 06 67 W USB-PD charger starved the board
→ PD needs a handshake chip → plain 5 V / 2 A adapter
- 07 Relay dead — pump never switched
→ Split breadboard rails → bridge + to +, - to -
- 08 temp always 0 °C
→ DHT11 on wrong pin → GPIO 4 + shared GND
- 09 Firebase “spurts” — 13 calls/s
→ One bundled call per second
- 10 Compile error: missing terminating quote
→ Copy-paste corruption → re-download the file



Why we show the failures

A demo that never broke teaches you nothing about the builders. The 17-calls bug, the USB-PD starvation, the FPC ribbon — each one is a real debugging story. Judges ask about the hard parts. We have



The pattern in every fix

Measure the symptom. Find the systemic cause — not the patch. Rebooting? It was the data flow. Not reading? It was power. That discipline is the actual engineering.



What we'd tell next year's team

Start with the wiring diagram. Test the power rail first. Never debug the network until the watchdog is fed. And when a charger “doesn't work”, check the handshake, not the wattage.

₹1,890. Everything. Yes, Really.

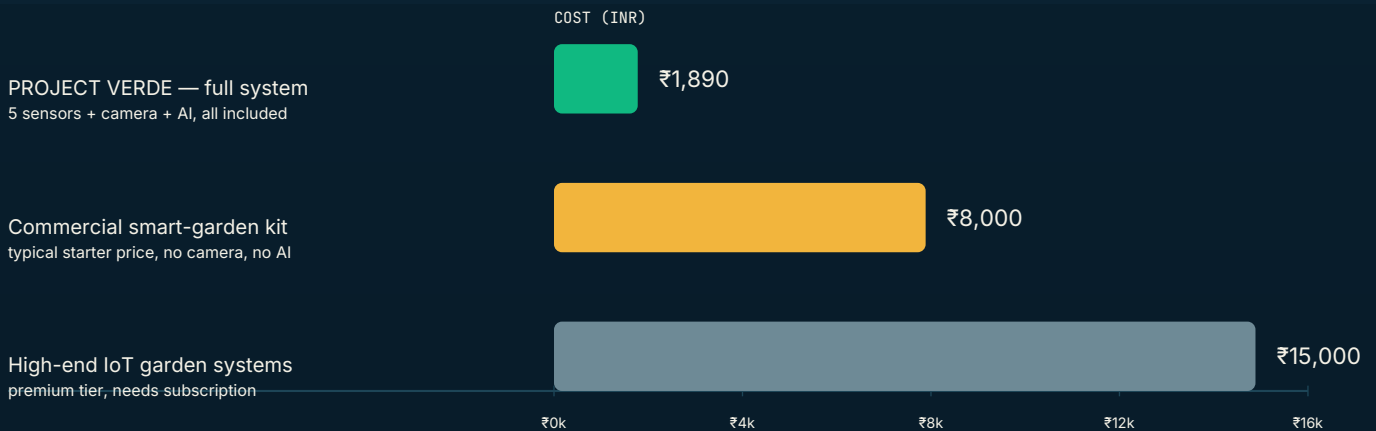
The full system — hardware, power, mechanics, and every API on free tiers.

CATEGORY	WHAT'S INSIDE	INR
Electronics	ESP32 · ESP32-CAM · 5 sensors · relay · pump · UV LED	1,320
Power & protection	5 V / 2 A adapter · 1000 µF cap · 1N4007 diode	220
Mechanical	breadboard · wires · enclosure	350
Software & APIs	all free tiers — Firebase, OWM, crop.health, Gemini, OpenRouter	0

TOTAL

≈ ₹1,890 (≈ \$23 USD)

The chart below is the whole point: ₹1,890 for the complete system — camera, five sensors, AI, the works — versus ₹8,000+ for a starter kit that still can't see the plant.



Low-power by design

power-gated 15 ms sensor reads, non-blocking scheduler — runs happily on a phone adapter.



Solar-ready

the roadmap adds a 12 V panel + charge controller + battery for off-grid autonomy.



Free-tier cloud

Firebase, weather, and AI all on free tiers — running cost ≈ ₹0/month.



Repairable by students

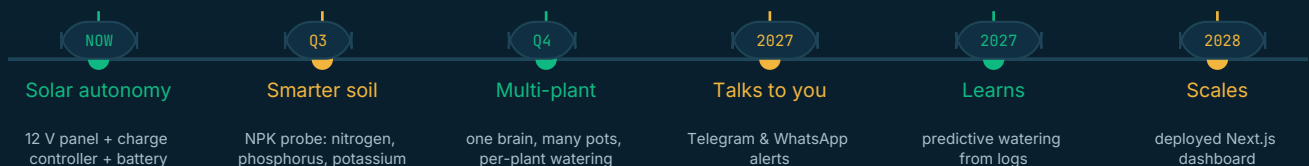
no proprietary parts, no black boxes — every wire is traceable on the schematic.

Future Scope: From One Pot to a Campus

Solar, smarter soil, and plants that message you. The architecture is ready for all of it.



Where this is heading — a solar-powered smart greenhouse, at dusk, watching its own garden.



- Solar autonomy — 12 V panel + charge controller + battery. The pot waters itself off-grid.
- NPK soil probe — nitrogen, phosphorus, potassium. From “is it wet?” to “what does it need?”
- Multi-plant zones — one brain, many pots, per-plant watering schedules.
- Telegram & WhatsApp alerts — “your basil is thirsty” arrives as a chat message.
- Predictive watering — the moisture logs train a model that waters **before** the wilt.
- Deployed dashboard — a scaffolded Next.js app ready to grow beyond the single file.

The hardware already earns the roadmap: power-gated sensors, tank protection, and a watchdog that survives 10-minute runs. Scaling Verde is a software problem — and software is the cheap part.

The 3-Minute Judge Script

Six stops. Practice it twice. The hardware and the app do the rest.

0:00

The hook

"This is a plant that waters itself — and it can tell us when it's sick." Point at the dashboard: 8 live tiles, already moving.

0:30

The brain

Open the pot. Show the ESP32 and five sensors. "It reads soil, air, light and the water tank — every second."

1:00

The AUTO moment

Dip the probe in water — the moisture tile climbs. "Watch the pump logic: below 35%, tank safe, no rain → water."

1:30

The eyes

Tap CAPTURE in Plant Doctor. Count to two — the photo appears. "That's an ESP32-CAM, end to end, under 2 seconds."

2:00

The AI

"What's wrong with this leaf?" — the diagnosis card and AI chat answer together. Mention the 94% on our test leaf.

2:30

The honesty

"And here's the bug that nearly killed it — 17 cloud calls a second, reboots, the works. We fixed it with one bundled call." Then: the cost — ₹1,890.

Practice the transitions. The system runs itself — the tour is just pointing at the truth. **Good luck. Not that you'll need it.**

IN CLOSING

Two Students. One Watered Idea.

Project Verde is complete, tested, and demo-ready.



We didn't build a gadget. We built a plant that can finally speak.

— Aarav & Anuj, Class X

Five sensors, two microcontrollers, one honest database, one beautiful web app, and four APIs that never once billed us. The system waters, watches, diagnoses, and talks — for less than the price of a fancy dinner.

Thank you for reading — and for judging. The demo is live on the table next to us. The plant is ready. **So are we.**



The builders

Aarav Choudhary & Anuj — Class X, DAV — built, broke, fixed, and shipped every part



The numbers

₹1,890 total · 17→2 calls/s · 94% diagnosis · 13/13 tests · 8 MHz XCLK



The system

ESP32 brain · ESP32-CAM eyes · 5 sensors · 2 actuators · Firebase · single-file web app



SCAN THE LIVE DATA

verde-tech-haha — Firebase feed
(swap in your hosted demo URL)